

Lesson 12: Count and for_each

Learning Objectives

- Use `count` to create multiple resources from a list
 - Use `for_each` to create resources from a map or set
 - Understand `count.index` and `each.key/each.value`
 - Choose between `count` and `for_each`
-

1. The `count` Meta-Argument

`count` creates a specified number of identical resources, differentiated by an index:

```
resource "aws_sqs_queue" "main" {
  count = 3
  name  = "queue-${count.index}"
}
```

This creates three queues: `queue-0`, `queue-1`, `queue-2`.

Accessing `count.index`

Each instance is referenced by its index:

```
resource "aws_sqs_queue" "main" {
  count = 3
  name  = "queue-${count.index}"
}

# Reference a specific instance
# aws_sqs_queue.main[0] → "queue-0"
# aws_sqs_queue.main[1] → "queue-1"
# aws_sqs_queue.main[2] → "queue-2"
```

Count with List Variable

```
variable "queue_names" {
  description = "Names for SQS queues"
  type        = list(string)
  default     = ["orders", "billing", "shipping"]
}

resource "aws_sqs_queue" "main" {
  count = length(var.queue_names)
```

```
name = "queue-${var.queue_names[count.index]}"
}
```

Creates: `queue-orders`, `queue-billing`, `queue-shipping`.

Conditional Count

Use a conditional expression to control whether a resource is created at all:

```
variable "create_bucket" {
  description = "Whether to create an S3 bucket"
  type        = bool
  default     = true
}
resource "aws_s3_bucket" "main" {
  count = var.create_bucket ? 1 : 0
  bucket = "conditional-bucket"
}
```

When `var.create_bucket` is `false`, count is `0` and no bucket is created. Access it with:

```
aws_s3_bucket.main[0].id
```

Resources created with `count` are always a **list** — even when count = 1.

2. The `for_each` Meta-Argument

`for_each` creates resources from a map or set of strings, differentiated by key:

```
resource "aws_sqs_queue" "main" {
  for_each = toset(["orders", "billing", "shipping"])
  name     = "queue-${each.key}"
}
```

Accessing `each.key` and `each.value`

- For a **set**: `each.key == each.value` (same value)
- For a **map**: `each.key` is the map key, `each.value` is the map value

```
variable "queue_configs" {
  description = "Queue configurations"
  type        = map(string)
  default = {
```

```
    orders    = "high"
    billing    = "critical"
    shipping   = "medium"
  }
}

resource "aws_sqs_queue" "main" {
  for_each = var.queue_configs
  name      = "queue-${each.key}"

  tags = {
    Priority = each.value
  }
}
```

Creates:

- `queue-orders` with tag `Priority = high`
- `queue-billing` with tag `Priority = critical`
- `queue-shipping` with tag `Priority = medium`

for_each with Complex Types

```
variable "buckets" {
  description = "Bucket configurations"
  type = map(object({
    versioning = bool
    region      = string
  }))
  default = {
    logs = {
      versioning = true
      region      = "us-east-1"
    }
    backup = {
      versioning = false
      region      = "eu-west-1"
    }
  }
}

resource "aws_s3_bucket" "main" {
  for_each = var.buckets
  bucket    = "bucket-${each.key}"
  region    = each.value.region
}
```

3. Count vs for_each

	count	for_each
Input type	Number	Set or map
Instance keys	Integer index (0, 1, 2)	String keys (from set/map)
Resource address	<code>resource.name[0]</code>	<code>resource.name["key"]</code>
Removal of middle item	Shifts all subsequent indices (can cause unexpected changes)	Only removes that specific key (safe)
Conditional creation	<code>count = 0</code> to skip	Not idiomatic
Use when	Creating N identical resources	Named resources with different config

When to use each:

Use `count` when:

- You need N identical resources
- You're using a conditional (`count = 0` to skip)
- The resources only differ by index

Use `for_each` when:

- Resources have distinct names/keys
- Removing an item from the middle must not shift other items
- You have a map of configurations with different values

4. Referencing Instances

count

```
# All instances as a list
aws_sqs_queue.main[*].id
# Returns: ["queue-0", "queue-1", "queue-2"]

# Specific instance
aws_sqs_queue.main[0].id
# Returns: "queue-0"

# Count in outputs
output "queue_ids" {
  value = aws_sqs_queue.main[*].id
}

output "first_queue" {
  value = aws_sqs_queue.main[0].id
}
```

for_each

```
# All instances as a map
aws_sqs_queue.main[*].id
# Returns: {"orders" = "queue-orders", "billing" = "queue-billing"}

# Specific instance
aws_sqs_queue.main["orders"].id
# Returns: "queue-orders"

# Map of values with splat
output "queue_arns" {
  value = { for k, q in aws_sqs_queue.main : k => q.arn }
}
```

5. Complete Example

```
# variables.tf
variable "environments" {
  description = "Environments to create queues for"
  type        = list(string)
  default     = ["dev", "qa", "prod"]
}

variable "queue_configs" {
  description = "Queue name -> delay mapping"
  type        = map(number)
  default = {
    orders = 0
    billing = 5
  }
}
```

```
        shipping = 10
    }
}

variable "create_extra_queue" {
    description = "Whether to create an extra queue"
    type        = bool
    default     = false
}

# main.tf
# Using count with list
resource "aws_sqs_queue" "env_queues" {
    count = length(var.environments)
    name  = "${var.environments[count.index]}-queue"

    tags = {
        Environment = var.environments[count.index]
    }
}

# Using for_each with map
resource "aws_sqs_queue" "service_queues" {
    for_each      = var.queue_configs
    name          = "${each.key}-queue"
    delay_seconds = each.value

    tags = {
        Service = each.key
    }
}

# Conditional count
resource "aws_sqs_queue" "extra" {
    count = var.create_extra_queue ? 1 : 0
    name  = "extra-queue"
}

# outputs.tf
output "env_queue_ids" {
    value = aws_sqs_queue.env_queues[*].id
}

output "service_queue_arns" {
    value = { for k, q in aws_sqs_queue.service_queues : k => q.arn }
}

output "extra_queue_id" {
    value = var.create_extra_queue ? aws_sqs_queue.extra[0].id : null
}
```

6. Key Takeaways

- `count` takes a number and creates indexed resources (`[0]`, `[1]`, etc.)
- `for_each` takes a set or map and creates keyed resources (`["key"]`)
- `count.index` gives the current index inside a `count` block
- `each.key` and `each.value` are available inside a `for_each` block
- Use `for_each` when items have stable keys (safer for removal)
- Use `count` for conditional resources and N identical copies
- Resources with `count` are a **list**; with `for_each` they are a **map**
- Splat expressions (`[*]`) collect all instances

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer – Eyuel M.